
VGR: Visual Grounded Reasoning

Jiacong Wang^{1,2*} Zijian Kang^{2*} Haochen Wang^{1,2*} Haiyong Jiang¹ Jiawen Li²
Bohong Wu² Ya Wang² Jiao Ran² Xiao Liang^{2†} Chao Feng^{2‡} Jun Xiao^{1‡}

¹School of Artificial Intelligence, University of Chinese Academy of Sciences

²ByteDance Inc.

*Equal Contribution. †Project Lead. ‡Corresponding Authors.

<https://huggingface.co/BytedanceDouyinContent/VGR>

Abstract

In the field of multimodal chain-of-thought (CoT) reasoning, existing approaches predominantly rely on reasoning on pure language space, which inherently suffers from language bias and is largely confined to math or science domains. This narrow focus limits their ability to handle complex visual reasoning tasks that demand comprehensive understanding of image details. To address these limitations, this paper introduces VGR, a novel reasoning multimodal large language model (MLLM) with enhanced fine-grained visual perception capabilities. Unlike traditional MLLMs that answer the question or reasoning solely on the language space, our VGR first detects relevant regions that may help to solve problems, and then provides precise answers based on replayed image regions. To achieve this, we conduct a large-scale SFT dataset called VGR-SFT that contains reasoning data with mixed vision grounding and language deduction. The inference pipeline of VGR allows the model to choose bounding boxes for visual reference and a replay stage is introduced to integrate the corresponding regions into the reasoning process, enhancing multimodal comprehension. Experiments on the LLaVA-NeXT-7B baseline show that VGR achieves superior performance on multi-modal benchmarks requiring comprehensive image detail understanding. Compared to the baseline, VGR uses only 30% of the image token count while delivering scores of +4.1 on MMStar, +7.1 on AI2D, and a +12.9 improvement on ChartQA.

1 Introduction

Large language models (LLMs) have demonstrated remarkable reasoning capabilities, particularly in complex problem-solving scenarios such as mathematical deduction and scientific analysis. Systems like OpenAI-o1 [33] and DeepSeek-R1 [12] exemplify this progress, achieving state-of-the-art performance on benchmarks requiring logical inference and algorithmic thinking, where the crux seems to be large-scale Reinforcement Learning (RL) [42] with verifiable rewards [39].

Recent advancements in multimodal reasoning have sought to extend these capabilities to vision-language tasks, often by distilling knowledge from powerful LLMs into multimodal architectures [14, 7, 54, 46, 6, 36, 37]. While promising results have emerged in math and science domains, studies consistently reveal a critical limitation: language bias [16, 49, 53], *i.e.*, over-reliance on linguistic priors leads to systematic performance drops in *perception-heavy* tasks.

To bridge this limitation, we propose **Visual Grounded Reasoning (VGR)**. Instead of reasoning solely in the language space, we argue that models should perform targeted visual analysis to identify key regions of interest directly relevant to the question during reasoning. VGR extends the conventional text-only chain of thought to the multi-model thinking trace, *allowing the model to selectively*

Correspondence to chaofeng.zz@bytedance.com and xiaojun@ucas.ac.cn

retrieve visual details on-demand, thereby enhancing the accuracy and interpretability of multi-modal reasoning. Specifically, we design a novel *self-driven* selective visual replay method to retrieve and replay the visual memory. The selective replay is controlled by the model with a form of predefined special signal: when the model requires visual grounding during reasoning, it generates a replay signal, prompting VGR to fetch corresponding visual tokens from the feature pool to augment its reasoning process. The feature pool is constructed from visual representations of high-resolution crops, and a pooling strategy is adopted to further enhance efficiency.

To learn this format of instruction, we further contribute a new reasoning dataset embedded with visual clues, marking the first attempt to explicitly model visual region attention in multi-modal reasoning. Unlike prior works that either rely on text-only chains of thought [53, 49] for multi-modal tasks or enforce rigid multi-turn interactions [50, 38, 34, 52], our dataset empowers models to autonomously attend to arbitrary visual regions during reasoning. Notably, all grounding areas in the dataset are voluntarily generated by the model itself, avoiding manual annotation bias. To construct this dataset, we first use an existing model to generate a cold-start dataset, which is then refined via a rejection sampling pipeline and further expanded using annotations from a custom-trained annotation model.

We conduct experiments on a novel framework designed to maximize information extraction from limited visual tokens. Experimental results using identical visual encoders and base LLMs show that our method achieves a performance score on the MMStar [4] and ChartQA [29] benchmark which outperforms the baseline LLaVA-Next by 6.4 and 14.1, while utilizing only 0.3 $\times$ visual tokens. These findings not only underscore the efficacy of selective feature replay in mitigating token redundancy but also establish a new paradigm for enhancing computational efficiency in MLLMs.

In summary, our contributions are three folds:

- We introduce VGR, a new visual reasoning framework for MLLM, which enables the model to selectively attend to visual content during inference, enhancing reasoning accuracy with fine-grained visual details.
- We build the reasoning data with visual cues for the first time, the dataset empowers models to freely attend to arbitrary visual regions during reasoning, contrasting with prior works relying on text-only chains of thought or rigid interactions.
- Extensive experiments on VGR demonstrate that our model outperforms the LLaVA-Next baseline in downstream tasks while using only 0.3 $\times$ the number of image tokens. As the quantity of image tokens increases, this performance gap becomes even more pronounced.

2 Related Works

Multimodal Large Language Models. Pioneering MLLM frameworks like Flamingo [1] and BLIP-2 [20] establish foundational architectures for cross-modal understanding using cross-attention. Subsequently, LLaVA [24] emerged as a more efficient, scalable, and modular framework, combining a vision encoder with a large language model through a simple linear projection layer. Its instruction-tuning paradigm demonstrated competitive performance, emphasizing the power of aligned vision-language supervision. Building on this idea, recent advancements [25, 48, 3, 44, 45, 28, 51, 56, 19, 47, 10, 11] push the boundaries of efficiency, scalability, and task complexity. For instance, Qwen2.5-VL [3] integrates dynamic resolution, and InternVL3 [56] emphasizes the importance of larger-scale native multimodal pretraining. These models serve as strong baselines for a variety of real-world applications.

Reasoning MLLMs. The groundbreaking success of advanced reasoning LLMs like OpenAI-o1 [33] and DeepSeek-R1 [12] has inspired efforts to extend such capabilities into multimodal domains. Prior attempts [49, 16] found that CoT prompting even brings performance degradation for perception-heavy tasks due to the accumulation of language bias. Therefore, current approaches mainly focus on incentivizing MLLMs to solve difficult math and science problems with image inputs. For instance, Vision-R1 [14] first leverages MLLMs to generate detailed captions for provided images and then queries DeepSeek-R1 [12] to obtain a dataset for cold initialization. Other approaches, such as VLM-R1 [40] and Visual-RFT [27], directly adopt GRPO [39] to open-ended visual grounding, where they found RL performs much better than SFT. This paper presents an alternative methodological approach that complements existing perspectives. Our framework seeks to incentivize the “grounding-then-answering” capability in MLLMs, requiring the model to systematically develop two critical

competencies: (1) frequent autonomous selection of task-relevant image regions through deliberate focus mechanisms, and (2) contextualized reasoning based on these visually grounded observations.

3 Visual Grounded Reasoning

In this section, we elaborate on the framework and model of VGR, in Figure 1. To unlock the visual grounding reasoning capabilities, we introduce a novel selective feature replay mechanism, allowing the model to attend to arbitrary image regions by retrieving corresponding image tokens during the reasoning on the fly.

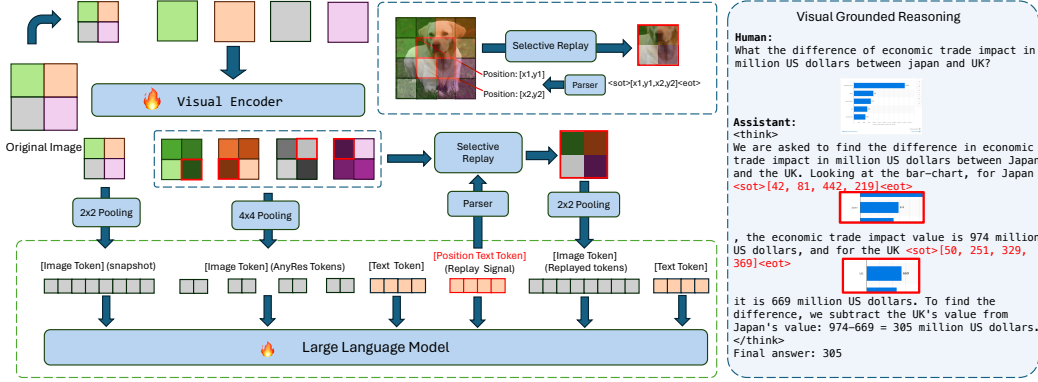


Figure 1: Overview framework of our method. In the left of the image, we crop the original image with AnyRes strategy to maintain the memory pool of visual details, when a replay signal is detected, VGR retrieves the image token from the memory pool, enrich visual clues in reasoning. In the right image, we show an example of VGR in action, VGR enables the MLLM to check the key area on-demand.

The selective replay module of VGR retrieves image tokens generated by the vision encoder and adapter. Leveraging LLaVA’s AnyRes approach for high-resolution image encoding, we first resize the input image to dimensions $H \times W$ where H and W are divisible by $p = 336$. The resized image $\mathbf{P} \in \mathbb{R}^{H \times W \times 3}$ is then partitioned into non-overlapping $p \times p$ patches:

$$\mathbf{P}_{ij} = \mathbf{P}[p * i : p * (i + 1), p * j : p * (j + 1)]. \quad (1)$$

The corresponding image tokens are processed by the vision-encoder and adapter, yielding token embeddings in the language space:

$$\mathbf{T}_{i,j} = \mathcal{F}_{adapter}(\mathcal{F}_{vision}(\mathbf{P}_{ij})) \in \mathbb{R}^{\frac{p}{s} \times \frac{p}{s} \times c}, \quad (2)$$

where s denotes the size of the vision patch and c denotes the channel number of latent features. Like in LLaVA, the image tokens from each crop are flattened to a 1D sequence and fed in the LLM separately. We further concatenate the feature of each patch representation to a unified image feature $\mathbf{S} \in \mathbb{R}^{\frac{H}{s} \times \frac{W}{s} \times c}$ for later use.

The replay mechanism relies on fine-grained visual feature for retrieval, to preserve high-resolution visual details while maintaining training and inference efficiency, we propose an expand-then-compress strategy. Specifically, we scale up the maximum crop count of LLaVA’s AnyRes approach from 4 to 16 patches and introduce a vision feature compression layer using 2D pooling. To balance resolution and computational cost, we further adopt 2×2 pooling for snapshot compression and 4×4 pooling for high-resolution AnyRes token compression empirically.

Compared to the baseline, which employs maximum 2,880 tokens per image (576 tokens per shot, including 1 snapshot image and 4 AnyRes crops), VGR achieves superior efficiency. Our method uses only 144 tokens for the snapshot and a maximum of 720 tokens for high-resolution crops, reducing token usage by 70% while expanding supported resolutions by $5\times$. This design guarantees VGR to maintain fine-grained visual information for retrieval while lowering computational overhead.

To enable the MLLM selectively attend to specific visual regions, we introduce a replay control signal for the model. Each replay region is defined via a grounding area notation: $\langle \text{cot} \rangle [x1, y1, x2,$

$y_2]\langle\text{eot}\rangle$, where $[x_1, y_1]$ denotes the top-left corner and $[x_2, y_2]$ the bottom-right corner of the region. The visual tokens will be retrieved once such signal is detected. The MLLM is encouraged to generate these signals during inference in demand to extend visual clues.

During inference, VGR monitors the model output and, upon detecting signal token $\langle\text{eot}\rangle$, parses the preceding content to extract the region coordinates. If valid, the model retrieves image tokens corresponding to this region from the feature map $\mathbf{S}$ and appends them after the control signal. Specifically, for a region defined by coordinates (x_1, y_1) and (x_2, y_2) , the feature replay module extracts the corresponding feature patch $\mathbf{R}_{x_1, y_1, x_2, y_2}$ as:

$$\mathbf{R}_{x_1, y_1, x_2, y_2} = \mathbf{S} [\lfloor y_1/s + 0.5 \rfloor : \lfloor y_2/s + 0.5 \rfloor, \lfloor x_1/s + 0.5 \rfloor : \lfloor x_2/s + 0.5 \rfloor], \quad (3)$$

where $\lfloor \cdot + 0.5 \rfloor$ denotes element-wise rounding to the nearest integer. The extracted feature map $\mathbf{R}_{x_1, y_1, x_2, y_2}$ is then down-sampled with 2×2 pooling and flattened into a 1D token sequence. They are fed into the LLM immediately following the replay signal token.

Implementing supervision for the selective replay is straightforward. We simply add the retrieved image tokens $\mathbf{R}_{x_1, y_1, x_2, y_2}$ to the training sequence after the replay signal and optimize the model with the standard supervised fine-tuning. The signal tokens as well as text tokens are supervised with cross-entropy loss, while all image tokens (both from the original input and the replay regions) are excluded from the loss computation. To further enhance the model’s region selection capability, we introduce an auxiliary detection loss that encourages accurate area predictions.

The detection loss is important because boxes are actually represented as numerical coordinates, $\mathcal{L}_{\text{det}}$ operates as a straightforward regression task to precisely align spatial locations, since cross-entropy on tokenized boxes may struggle with quantization errors and discontinuous predictions. Therefore, combining both allows the model to leverage continuous regression for accurate localization. Specifically, the detection loss is a combination of ℓ_1 loss and GIoU loss:

$$\mathcal{L}_{\text{det}} = \ell_1 + \beta \ell_{\text{GIoU}}, \quad (4)$$

where ℓ_1 Loss measures the absolute difference between predicted bounding box coordinates and ground truth. We set $\beta = 2$ following common practices. For a bounding box parameterized by center coordinates (x_c, y_c) , width w , and height h , the formula is:

$$\ell_1 = |\hat{x}_c - x_c| + |\hat{y}_c - y_c| + |\hat{w} - w| + |\hat{h} - h|, \quad (5)$$

where $\hat{x}_c, \hat{y}_c, \hat{w}, \hat{h}$ are predictions. GIoU loss is computed by:

$$\ell_{\text{GIoU}} = 1 - \left(\frac{\text{InterArea}}{\text{UnionArea}} - \frac{C - \text{UnionArea}}{C} \right), \quad (6)$$

where C is the smallest box enclosing both predicted and ground truth boxes:

$$C = (x_C^2 - x_C^1) \cdot (y_C^2 - y_C^1), \quad (7)$$

where $x_C^1 = \min(x_1, \hat{x}_1)$, $x_C^2 = \max(x_2, \hat{x}_2)$, $y_C^1 = \min(y_1, \hat{y}_1)$, $y_C^2 = \max(y_2, \hat{y}_2)$. The detection head we utilized is a small MLP that maps the hidden states of $\langle\text{eot}\rangle$ to a 4-dimensional box.

4 Visual Reasoning Data Curation

VGR learns to the visual reasoning through our reasoning data with selective replay, with the proposed three-stage data construction pipeline as shown in Figure 2. The cold start data is generated with an existing large instruction model and further refined with reject sampling. Then, we train an annotation model to annotate data from more domains. Our framework extends the conventional text-only reasoning chain to a multimodal reasoning chain for the first time.

4.1 Cold-start with Instruction Model

The initial instruction data with replay capabilities is generated using an existing vision-language model. Specifically, given an image and a corresponding question, the model is prompted to generate both a reasoning chain and an answer. Concurrently, we require the model to localize all key regions

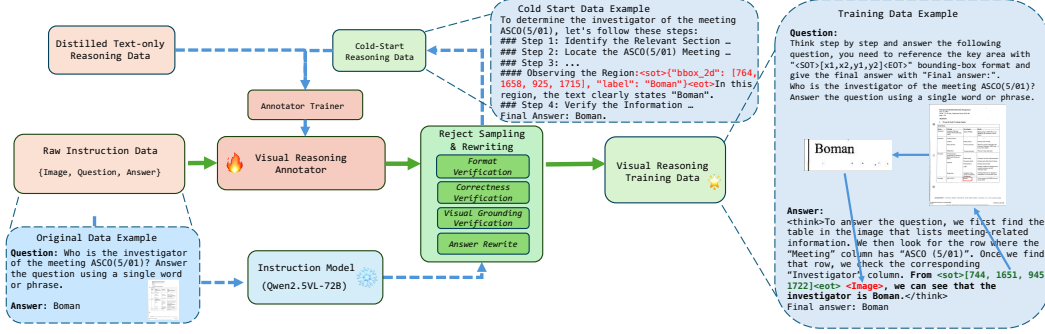


Figure 2: Overview framework of our data pipeline. The blue arrow line indicates the cold-start data curation pipeline for the annotator and the green line indicates the data pipeline for training data.

in the image relevant to the answer and explicitly reference these regions before describing their content. These key regions are designated as replay areas during training.

We select Qwen2.5-VL-72B [43] as the cold-start model due to its exceptional instruction-following capabilities, output diversity, and strong performance in both object detection and visual reasoning tasks. To standardize the annotation format, we prompt the model to encode detection results in JSON, which includes bounding boxes and semantic labels for each key region.

4.2 Reject Sampling

Following the recent advances in RL [12], we propose a similar reject sampling pipeline for valid data selection. First, we employ **Format Verification** to ensure answer parseability. This involves two checks: (1) verifying that answers can be extracted by locating the designated “Final Answer” section; (2) ensuring bounding boxes and labels are formatted in valid JSON. Next, **Correctness Verification** assesses the accuracy of answers derived from reasoning chains. For closed-ended tasks (e.g., OCR, MCQ), we use ANLS (Average Normalized Levenshtein Similarity) to quantify correctness by comparing generated answers with ground truths. For open-ended tasks, we leverage a commercial model API to semantically align reasoning chains with reference answers. Incorrect answers are discarded, while inaccurate ones undergo rewriting: the final answer is replaced with ground truth, and reasoning chains for open-ended tasks are iteratively revised for coherence. Finally, **Visual Grounding Verification** validates the correctness of replay areas. During data preparation, each replay area is annotated with a bounding box and semantic label. We crop these areas and use a commercial model to check alignment between cropped content and annotated labels. Additionally, we intentionally expand bounding box areas to encourage the trained model to retain contextual information during reasoning, enhancing its ability to handle complex visual-semantic dependencies.

4.3 Data Scaling with Annotation Model

During the reject sampling, we notice the cold-start data generated by existing instruction-tuned models exhibits a high rejection rate and slow generation speed. To address these limitations, we train an annotation model using a small subset of the cold-start data that survived the reject sampling pipeline. Empirically, we initialize our model with the recently released InternVL3-14B [56] and further augment it with cold-start data and pure text data from the Open-R1 distilled dataset [8]. This augmentation strategy leverages reasoning patterns learned from large-scale text data, which we found can generalize effectively to visual reasoning tasks. The annotation model improves the pass rate from 14% to 40% and significantly speedup annotation.

4.3.1 Training Data

The training data is derived from the annotated samples passing through the reject sampling pipeline. To ensure consistency with the LLaVA baseline [23], we curate training data **VGR-SFT** from LLaVA’s official training sets, with the processed dataset and its statistics detailed in Table 1. We further employ the online model to revise the reasoning chains, enhancing reasoning robustness. We prompted the model to strictly aligns the data with our predefined template while eliminating

ambiguous or redundant content. The refined data is subsequently utilized to train the final reasoning model, ensuring its capacity to generate structured and coherent responses.

Table 1: The number of data generated from each dataset.

Method	Data Size	Data Type
AI2D [18]	12.5k	ScienceQA
LLaVA-COCO [23]	12.3k	General VQA
GQA [15]	39.2k	General VQA
ChartQA [29]	11.2k	OCR
DVQA [17]	25.2k	OCR
DocVQA [30]	6.0k	OCR
OCRVQA [32]	51.6k	OCR
Total	158.1k	-

5 Experiments

5.1 Experiment Settings

Our VGR is built upon LLaVA-NeXT [26], a flexible and strong baseline for multimodal comprehension. The visual encoder is CLIP-ViT-L/14@336 [35] and the base LLM is Vicuna-v1.5 series [5], including 7B and 13B versions. Following [26], VGR has two training procedures: pre-training and supervised fine-tuning. The pre-training data is LLaVA-558K [25], while the fine-tuning data is the combination of LLaVA-NeXT-770K [26] and our self-constructed 158K data. Notably, to ensure a fair comparison, all datasets constructed are derived from the original SFT data of LLaVA-Next without introducing any additional data. The LR for pre-training stage is set to $1e-5$ and $2e-5$ for fine-tuning stage with Vicuna-7B. We set the learning rate of ViT to 1/10 of the base learning rate follow the LLaVA-NeXT’s setting.

5.2 Comparison with existing methods

We compare our VGR with a wide range of alternatives, including Qwen-VL-Chat [2], Visual CoT [38], DeepSeek-VL-7B [28], LLaVA-v1.5-7B [22] and LLaVA-NeXT-7B [26]. As demonstrated in Table 2, our VGR manages to perform the best under most cases, especially on benchmarks requiring fine-grained comprehension for high-resolution images. Moreover, considering the average number of visual tokens, VGR achieves better performance with $[N] \times \text{visual tokens}$ compared to the original LLaVA-NeXT [26], which implies that focusing the model on *specific regions* is much more effective than simply utilizing more visual tokens. As the quantity of image tokens increases, this performance gap becomes even more pronounced.

5.3 Ablation Studies

Ablations on different data formulations. The ablation study in Table 3 demonstrates that requiring the model to output *both* grounding boxes and reasoning procedures is critical for effective multimodal understanding. When either component is removed, whether by eliminating grounding cues (“w/o Grounding”) or disabling the reasoning process (“w/o Reasoning”), performance *consistently* degrades across multiple benchmark datasets. Resulting surface precise grounding and effective reasoning help each other.

Ablations on detection loss. In Table 4a, we study the effectiveness of the auxiliary detection loss. Since boxes are represented by floating-point coordinates normalized to the $[0, 1]$ range. $\mathcal{L}_{\text{det}}$ operates as a straightforward regression task to precisely align spatial locations, since cross-entropy on tokenized boxes may struggle with quantization errors and discontinuous predictions. Therefore, combining both allows the model to leverage continuous regression for accurate localization.

Ablations on feature replay. In Table 4b, we systematically evaluate the efficacy of feature replay through ablation studies. Results show that excluding feature replay where the model merely outputs regions-of-interest *without* incorporating corresponding image features into the LLM input sequence leads to significantly limited performance improvements. This highlights the critical gain from

Table 2: Comparison with existing vision-language models on various vision-language benchmarks, including MMStar [4]; ChartQA [29]; DocVQA [30]; TextVQA [41]; InfoQA [31]; AI2D [18]; RealWorldQA [9]; POPE [21]. Note that Sample represents the image token compression or down sample scheme used; Vtoken indicates the maximum image patch tokens. The top results are highlighted in **bold**. All results are derived from those reported in other papers and the official reproduction results from the LMMs-Eval [55]. Our results are obtained using LMMs-Eval. [†] indicates our reproduction with a maximum of 20 local images with LLaVA-NeXT [26] and feature pooling (2×2 for the base crop and 4×4 for local crops). The replay image features are also applied with 2×2 pooling.

Method	DownSample	#Vtoken	LLM	MMS*	Chart	Doc	Text	Info	AI2D	RWQA	POPE
Qwen-VL-Chat-7B [2]	Cross-Attn	1024	Qwen-7B	34.5	66.3	62.6	61.5	-	57.7	49.3	74.9
Visual CoT [38]	No	576	Vicuna-7B	-	22.8	49.3	66.9	-	-	-	86.5
DeepSeek-VL-7B [28]	Conv2D	576	DeepSeek-7B	40.5	59.1	-	64.9	-	65.3	54.2	85.6
LLaVA-v1.5-7B [22]	No	576	Vicuna-7B	33.1	18.2	28.1	46.1	25.8	54.8	54.8	85.9
LLaVA-NeXT-7B [26]	No	2880	Vicuna-7B	37.6	54.8	77.4	64.9	37.1	66.6	57.8	86.5
LLaVA-NeXT-7B [†] [26]	2×2 14×4	864	Vicuna-7B	37.2	58.7	70.2	60.5	34.7	68.5	56.8	87.8
VGR-7B	2×2 14×4	864	Vicuna-7B	41.7	67.7	73.7	63.9	39.8	73.7	59.8	88.2
VGR-7B	2×2 12×2	3024	Vicuna-7B	43.6	72.8	79.9	65.9	42.9	73.4	59.5	87.8

Table 3: **Ablations on different data formulations.** [†] indicates our reproduction with a maximum of 20 local images and feature pooling (2×2 for the base crop and 4×4 for local crops). “w/o Grounding” indicates that only the reasoning process is preserved without grounding and replay. “w/o Reasoning” means we remove the reasoning process.

Method	MMStar	ChartQA	DocVQA	TextVQA	InfoVQA	AI2D	RWQA	POPE
LLaVA-NeXT-7B [†] [26]	37.2	58.7	70.2	60.5	34.7	68.5	56.8	87.8
VGR-7B	41.7	67.7	73.7	63.9	39.8	73.7	59.8	88.2
w/o Grounding	39.7	66.2	73.2	63.0	39.3	72.7	60.6	87.5
w/o Reasoning	39.3	59.6	72.5	61.9	38.5	72.8	59.3	87.8

integrating image features of boundary regions into the reasoning process, as it enables the model to leverage fine-grained visual details for more accurate predictions.

Ablations on the number of maximum local crops. Since our replayed features are directly derived from preceding local features, increasing the number of local crops could theoretically enhance feature richness. To validate this, we conducted experiments as shown in Table 4c, where empirical results demonstrate that using 16 crops yields significantly superior performance compared to fewer cropping strategies.

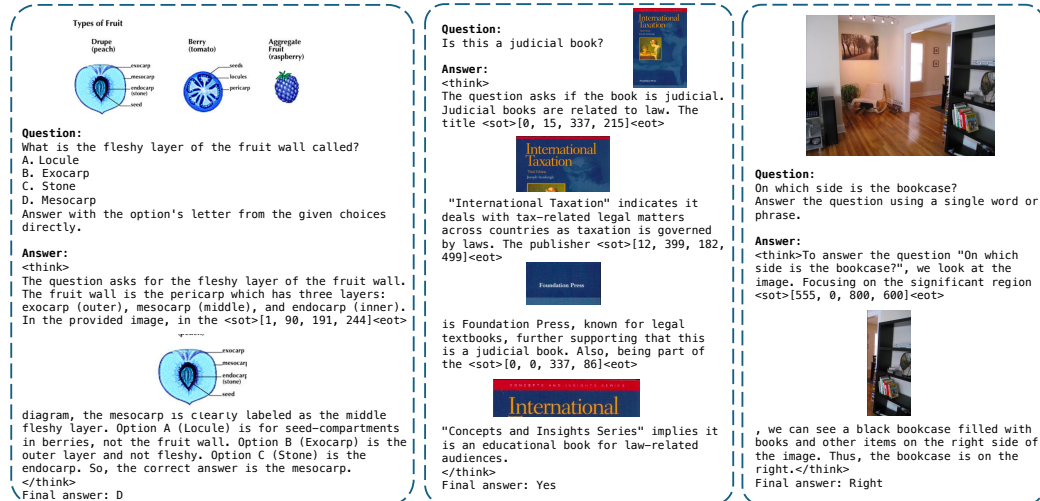


Figure 3: Example of training data in VGR-SFT.

Table 4: **Ablations on each component**, including (a) the introduction of detection loss, (b) whether to replay visual features after predicting bounding boxes, and (c) the type of reasoning data. By default, we enable detection loss and feature replay with a maximum of 20 local crops.

(a) Detection loss.				(b) Feature replay.			(c) Reasoning Data Type.			
$\mathcal{L}_{\text{det}}$	MMStar	ChartQA	DocVQA	Replay	MMStar	ChartQA	Type	MMStar	ChartQA	AI2D
–	39.8	65.5	72.8	–	39.7	66.2	Long	40.7	64.5	71.7
✓	41.7	67.7	73.7	✓	41.7	67.7	Short	41.7	67.7	73.7

Table 5: **Ablations on different CoT data**, where we combine our datasets with the original SFT dataset and fine-tune the LLaVA-NeXT-7B with a maximum of 20 local crops and pooling, while others require an extra post-training stage. [†] indicates our reproduction with a maximum of 20 local images with LLaVA-NeXT and feature pooling

Data	SFT	Post-Train	MMStar	ChartQA	DocVQA	ScienceQA
LLaVA-NeXT [†] [26]	770K	–	37.2	58.7	70.2	70.3
LLaVA-CoT [†] [53]	770K	100K	39.6	58.8	64.4	76.5
MMPR [†] [49]	770K	660K	40.7	55.1	68.3	82.1
VGR-7B	770K + 158K	–	41.7	67.7	73.7	70.4

Ablations on different CoT data. We compare the effectiveness of our reasoning data (which explicitly utilizes regions-of-interest) against vanilla reasoning datasets such as LLaVA-CoT [53] and MMPR [49]. As shown in Table 5, the model with region-of-interest guidance focuses more on relevant visual areas, leading to improved overall performance. In contrast, direct adoption of complex reasoning datasets like [53, 49] yields results even worse than the baseline. This may stem from the accumulation of language bias during multimodal reasoning, highlighting that appropriately integrating visual features of regions-of-interest significantly aids accurate inference.

Table 6: **Ablations on different pooling strategies.** Different pooling strides for each type of image are important. We utilize 2×2 for **Base** image feature and **Replayed** image feature, and 4×4 for **Local** images feature.

Base	Local	Replayed	#Crops	#Vtoken	MMStar	ChartQA	DocVQA	TextVQA	InfoQA
–	–	–	4	2880	37.2	58.7	70.2	60.5	34.7
2×2	4×4	2×2	4	288+20	37.5	53.4	52.0	57.0	30.1
2×2	4×4	2×2	20	864+100	41.7	67.7	73.7	63.9	39.8
2×2	2×2	2×2	20	3024+100	43.6	72.8	79.9	65.9	42.9

Ablations on different pooling strategies In Table 6, we investigate the trade-off between pooling performance by differentiating the pooling steps for base images, local images, and replayed images, as these components exhibit distinct levels of importance. The base image, which encapsulates the most comprehensive understanding of the entire visual content, demands a balance between global context and spatial detail. Local crops, while useful, often contain redundant information due to overlapping regions, justifying coarser pooling. Replayed images, however, represent specific regions of interest (RoIs) critical to task-solving and thus require finer-grained feature preservation compared to standard local crops. Empirically, the optimal configuration employs 2×2 pooling for both base and replayed images to retain critical details, while applying 4×4 pooling to local images to mitigate redundancy without significant information loss.

5.4 Exploring Direct Apply of Larger Image Cropping Strategies at Test Time.

We attempted to adopt a larger image cropping scheme during testing to generate more image tokens, while keeping the pooling strategy unchanged. The test results shown in Table 7 indicate that simply increasing the resolution does not directly enhance QA performance for some datasets, which is related to the intrinsic characteristics of the dataset, such as image distribution and native resolution. The additional tokens are calculated based on the average crop area in the data and extracted crop image features resolution. We set 64 for the max crop image number, in practice, most images in the fine-tuning data do not reach such resolution.

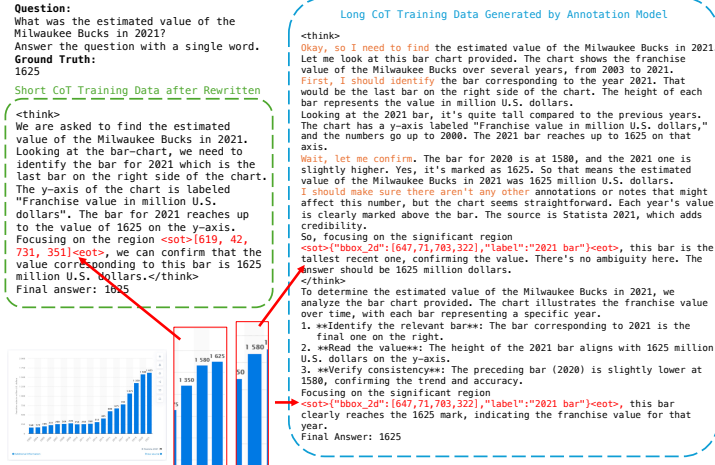


Figure 4: Example generated by our annotation model. We distill core information and the chain-of-thought from long redundant reasoning with reject sampling and rewriting.

Table 7: **Test Time Image Tokens Scaling.** We apply a larger image cropping scheme during testing to obtain more image tokens. Other setting is same as the Table 6.

Base	Local	Replayed	#Crops	#Vtoken	MMStar	ChartQA	DocVQA	TextVQA	InfoQA
2×2	4×4	2×2	20	864+100	41.7	67.7	73.7	63.9	39.8
2×2	4×4	2×2	64	2592+400	42.9	67.9	76.3	63.9	42.9

5.5 Qualitative Observation on Data Annotation

To illustrate the effectiveness and necessity of our data pipeline, we show examples from our training data, shown in Figure 3. To expose more details, data from annotation model and refine module in Figure 4. The right part of the figure contains an example generated by our annotation model. After training with cold-start data and complex reasoning data from DeepSeek-R1 [8, 12], the annotation model can generalize reasoning ability from text to multi-modal. However, this model still easily makes various mistakes, so our reject sampling pipeline and rewritten module are necessary to fix these issues. In this example, the reject sampling and checking module expands the bounding-box, aligns it with the correct ground truth, and enriches the context. The rewritten module removes duplicate bounding-boxes, reformats the document, and clarifies the data. Short and clean data are especially valuable for smaller-scale models like Vicuna [5] that only supports 4096 tokens. As shown in Table 4c, short clean data also performs better than long data in our experiments.

6 Conclusion

In this work, we propose VGR for enhanced multimodal comprehension. VGR enables MLLMs to reason on visual clues and selectively attend to informative regions on demand. To achieve this, we introduce a selective feature replay module, which allows the model to focus on crucial regions, thereby enhancing fine-grained comprehension—particularly for small regions in high-resolution inputs. We also curate a large-scale reasoning dataset, VGR-SFT, which for the first time integrates visual information into dense reasoning tasks. Extensive experiments on VGR demonstrate considerable improvements across multiple benchmarks, validating the effectiveness of our approach.

Discussion. Our method has several limitations that warrant future research. First, VGR is currently constrained to LLaVA [23] architecture, exploring stronger visual encoders and LLMs could further enhance performance. Another avenue is integrating reinforcement learning (RL). With cold-start initialization and RL training, a more generalized and diverse reasoning process may be achievable.

References

- [1] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katherine Millican, Malcolm Reynolds, et al. Flamingo: a visual language model for few-shot learning. *Advances in neural information processing systems*, 35: 23716–23736, 2022.
- [2] Jinze Bai, Shuai Bai, Shusheng Yang, Shijie Wang, Sinan Tan, Peng Wang, Junyang Lin, Chang Zhou, and Jingren Zhou. Qwen-vl: A versatile vision-language model for understanding, localization, text reading, and beyond. 2023.
- [3] Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibao Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, et al. Qwen2.5-vl technical report. *arXiv preprint arXiv:2502.13923*, 2025.
- [4] Lin Chen, Jinsong Li, Xiaoyi Dong, Pan Zhang, Yuhang Zang, Zehui Chen, Haodong Duan, Jiaqi Wang, Yu Qiao, Dahua Lin, et al. Are we on the right way for evaluating large vision-language models? *arXiv preprint arXiv:2403.20330*, 2024.
- [5] Wei-Lin Chiang, Zhuohan Li, Ziqing Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E Gonzalez, et al. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality. See <https://vicuna.lmsys.org> (accessed 14 April 2023), 2(3):6, 2023.
- [6] Hongyuan Dong, Jiawen Li, Bohong Wu, Jiacong Wang, Yuan Zhang, and Haoyuan Guo. Benchmarking and improving detail image caption. *arXiv preprint arXiv:2405.19092*, 2024.
- [7] Hongyuan Dong, Zijian Kang, Weijie Yin, Xiao Liang, Chao Feng, and Jiao Ran. Scalable vision language model training via high quality data curation. *arXiv preprint arXiv:2501.05952*, 2025.
- [8] Hugging Face. Open r1: A fully open reproduction of deepseek-r1, January 2025. URL <https://github.com/huggingface/open-r1>.
- [9] Grok. Grok-1.5 vision preview, 2024. URL <https://x.ai/blog/grok-1.5v>.
- [10] Xin Gu, Heng Fan, Yan Huang, Tiejian Luo, and Libo Zhang. Context-guided spatio-temporal video grounding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 18330–18339, 2024.
- [11] Xin Gu, Yaojie Shen, Chenxi Luo, Tiejian Luo, Yan Huang, Yuewei Lin, Heng Fan, and Libo Zhang. Knowing your target: Target-aware transformer makes better spatio-temporal video grounding. *arXiv preprint arXiv:2502.11168*, 2025.
- [12] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- [13] Dong Guo, Faming Wu, Feida Zhu, Fuxing Leng, Guang Shi, Haobin Chen, Haoqi Fan, Jian Wang, Jianyu Jiang, Jiawei Wang, et al. Seed1. 5-vl technical report. *arXiv preprint arXiv:2505.07062*, 2025.
- [14] Wenxuan Huang, Bohan Jia, Zijie Zhai, Shaosheng Cao, Zheyu Ye, Fei Zhao, Zhe Xu, Yao Hu, and Shaohui Lin. Vision-r1: Incentivizing reasoning capability in multimodal large language models. *arXiv preprint arXiv:2503.06749*, 2025.
- [15] Drew A Hudson and Christopher D Manning. Gqa: A new dataset for real-world visual reasoning and compositional question answering. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 6700–6709, 2019.
- [16] Dongzhi Jiang, Renrui Zhang, Ziyu Guo, Yanwei Li, Yu Qi, Xinyan Chen, Liuhui Wang, Jianhan Jin, Claire Guo, Shen Yan, et al. Mme-cot: Benchmarking chain-of-thought in large multimodal models for reasoning quality, robustness, and efficiency. *arXiv preprint arXiv:2502.09621*, 2025.

- [17] Kushal Kafle, Brian Price, Scott Cohen, and Christopher Kanan. Dvqa: Understanding data visualizations via question answering. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 5648–5656, 2018.
- [18] Aniruddha Kembhavi, Mike Salvato, Eric Kolve, Minjoon Seo, Hannaneh Hajishirzi, and Ali Farhadi. A diagram is worth a dozen images. In *Computer Vision–ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11–14, 2016, Proceedings, Part IV 14*, pages 235–251. Springer, 2016.
- [19] Weixian Lei, Jiacong Wang, Haochen Wang, Xiangtai Li, Jun Hao Liew, Jiashi Feng, and Zilong Huang. The scalability of simplicity: Empirical analysis of vision-language learning with a single transformer. *arXiv preprint arXiv:2504.10462*, 2025.
- [20] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *International conference on machine learning*, pages 19730–19742. PMLR, 2023.
- [21] Yifan Li, Yifan Du, Kun Zhou, Jinpeng Wang, Wayne Xin Zhao, and Ji-Rong Wen. Evaluating object hallucination in large vision-language models. *arXiv preprint arXiv:2305.10355*, 2023.
- [22] Haotian Liu, Chunyuan Li, Yuheng Li, and Yong Jae Lee. Improved baselines with visual instruction tuning, 2023.
- [23] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *NeurIPS*, 2023.
- [24] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. *Advances in neural information processing systems*, 36:34892–34916, 2023.
- [25] Haotian Liu, Chunyuan Li, Yuheng Li, and Yong Jae Lee. Improved baselines with visual instruction tuning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 26296–26306, 2024.
- [26] Haotian Liu, Chunyuan Li, Yuheng Li, Bo Li, Yuanhan Zhang, Sheng Shen, and Yong Jae Lee. Llava-next: Improved reasoning, ocr, and world knowledge, January 2024. URL <https://llava-v1.github.io/blog/2024-01-30-llava-next/>.
- [27] Ziyu Liu, Zeyi Sun, Yuhang Zang, Xiaoyi Dong, Yuhang Cao, Haodong Duan, Dahua Lin, and Jiaqi Wang. Visual-rft: Visual reinforcement fine-tuning. *arXiv preprint arXiv:2503.01785*, 2025.
- [28] Haoyu Lu, Wen Liu, Bo Zhang, Bingxuan Wang, Kai Dong, Bo Liu, Jingxiang Sun, Tongzheng Ren, Zhuoshu Li, Hao Yang, et al. Deepseek-vl: towards real-world vision-language understanding. *arXiv preprint arXiv:2403.05525*, 2024.
- [29] Ahmed Masry, Do Xuan Long, Jia Qing Tan, Shafiq Joty, and Enamul Hoque. Chartqa: A benchmark for question answering about charts with visual and logical reasoning. *arXiv preprint arXiv:2203.10244*, 2022.
- [30] Minesh Mathew, Dimosthenis Karatzas, and CV Jawahar. Docvqa: A dataset for vqa on document images. In *Proceedings of the IEEE/CVF winter conference on applications of computer vision*, pages 2200–2209, 2021.
- [31] Minesh Mathew, Viraj Bagal, Rubèn Tito, Dimosthenis Karatzas, Ernest Valveny, and CV Jawahar. Infographicvqa. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pages 1697–1706, 2022.
- [32] Anand Mishra, Shashank Shekhar, Ajeet Kumar Singh, and Anirban Chakraborty. Ocr-vqa: Visual question answering by reading text in images. In *2019 international conference on document analysis and recognition (ICDAR)*, pages 947–952. IEEE, 2019.
- [33] OpenAI. Openai-o1, 2024.

- [34] Ji Qi, Ming Ding, Weihang Wang, Yushi Bai, Qingsong Lv, Wenyi Hong, Bin Xu, Lei Hou, Juanzi Li, Yuxiao Dong, et al. Cogcom: Train large vision-language models diving into details through chain of manipulations. *arXiv preprint arXiv:2402.04236*, 2024.
- [35] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PMLR, 2021.
- [36] Zhongwei Ren, Zhicheng Huang, Yunchao Wei, Yao Zhao, Dongmei Fu, Jiashi Feng, and Xiaojie Jin. Pixellm: Pixel reasoning with large multimodal model. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 26374–26383, 2024.
- [37] Zhongwei Ren, Yunchao Wei, Xun Guo, Yao Zhao, Bingyi Kang, Jiashi Feng, and Xiaojie Jin. Videoworld: Exploring knowledge learning from unlabeled videos. *arXiv preprint arXiv:2501.09781*, 2025.
- [38] Hao Shao, Shengju Qian, Han Xiao, Guanglu Song, Zhuofan Zong, Letian Wang, Yu Liu, and Hongsheng Li. Visual cot: Advancing multi-modal language models with a comprehensive dataset and benchmark for chain-of-thought reasoning. *Advances in Neural Information Processing Systems*, 37:8612–8642, 2024.
- [39] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Y Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [40] Haozhan Shen, Peng Liu, Jingcheng Li, Chunxin Fang, Yibo Ma, Jiajia Liao, Qiaoli Shen, Zilun Zhang, Kangjia Zhao, Qianqian Zhang, et al. Vlm-r1: A stable and generalizable r1-style large vision-language model. *arXiv preprint arXiv:2504.07615*, 2025.
- [41] Amanpreet Singh, Vivek Natarajan, Meet Shah, Yu Jiang, Xinlei Chen, Dhruv Batra, Devi Parikh, and Marcus Rohrbach. Towards vqa models that can read. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 8317–8326, 2019.
- [42] Richard S Sutton, Andrew G Barto, et al. *Reinforcement learning: An introduction*, volume 1. MIT press Cambridge, 1998.
- [43] Qwen Team. Qwen2.5-vl, January 2025. URL <https://qwenlm.github.io/blog/qwen2.5-vl/>.
- [44] Haochen Wang, Anlin Zheng, Yucheng Zhao, Tiancai Wang, Zheng Ge, Xiangyu Zhang, and Zhaoxiang Zhang. Reconstructive visual instruction tuning. *arXiv preprint arXiv:2410.09575*, 2024.
- [45] Haochen Wang, Yucheng Zhao, Tiancai Wang, Haoqiang Fan, Xiangyu Zhang, and Zhaoxiang Zhang. Ross3d: Reconstructive visual instruction tuning with 3d-awareness. *arXiv preprint arXiv:2504.01901*, 2025.
- [46] Haozhe Wang, Chao Qu, Zuming Huang, Wei Chu, Fangzhen Lin, and Wenhui Chen. VI-rethinker: Incentivizing self-reflection of vision-language models with reinforcement learning. *arXiv preprint arXiv:2504.08837*, 2025.
- [47] Jiacong Wang, Bohong Wu, Haiyong Jiang, Zhou Xun, Xin Xiao, Haoyuan Guo, and Jun Xiao. World to code: Multi-modal data generation via self-instructed compositional captioning and filtering. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 4608–4623, 2024.
- [48] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, et al. Qwen2-vl: Enhancing vision-language model’s perception of the world at any resolution. *arXiv preprint arXiv:2409.12191*, 2024.
- [49] Weiyun Wang, Zhe Chen, Wenhui Wang, Yue Cao, Yangzhou Liu, Zhangwei Gao, Jinguo Zhu, Xizhou Zhu, Lewei Lu, Yu Qiao, et al. Enhancing the reasoning ability of multimodal large language models via mixed preference optimization. *arXiv preprint arXiv:2411.10442*, 2024.

- [50] Penghao Wu and Saining Xie. V?: Guided visual search as a core mechanism in multimodal llms. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 13084–13094, 2024.
- [51] Zhiyu Wu, Xiaokang Chen, Zizheng Pan, Xingchao Liu, Wen Liu, Damai Dai, Huazuo Gao, Yiyang Ma, Chengyue Wu, Bingxuan Wang, et al. Deepseek-vl2: Mixture-of-experts vision-language models for advanced multimodal understanding. *arXiv preprint arXiv:2412.10302*, 2024.
- [52] Xin Xiao, Bohong Wu, Jiacong Wang, Chunyuan Li, Haoyuan Guo, et al. Seeing the image: Prioritizing visual correlation by contrastive alignment. *Advances in Neural Information Processing Systems*, 37:30925–30950, 2024.
- [53] Guowei Xu, Peng Jin, Li Hao, Yibing Song, Lichao Sun, and Li Yuan. Llava-o1: Let vision language models reason step-by-step. *arXiv preprint arXiv:2411.10440*, 2024.
- [54] Yi Yang, Xiaoxuan He, Hongkun Pan, Xiyan Jiang, Yan Deng, Xingtao Yang, Haoyu Lu, Dacheng Yin, Fengyun Rao, Minfeng Zhu, et al. R1-onevision: Advancing generalized multimodal reasoning through cross-modal formalization. *arXiv preprint arXiv:2503.10615*, 2025.
- [55] Kaichen Zhang, Bo Li, Peiyuan Zhang, Fanyi Pu, Joshua Adrian Cahyono, Kairui Hu, Shuai Liu, Yuanhan Zhang, Jingkang Yang, Chunyuan Li, and Ziwei Liu. Lmms-eval: Reality check on the evaluation of large multimodal models, 2024. URL <https://arxiv.org/abs/2407.12772>.
- [56] Jinguo Zhu, Weiyun Wang, Zhe Chen, Zhaoyang Liu, Shenglong Ye, Lixin Gu, Yuchen Duan, Hao Tian, Weijie Su, Jie Shao, et al. Internvl3: Exploring advanced training and test-time recipes for open-source multimodal models. *arXiv preprint arXiv:2504.10479*, 2025.

A More Results of VGR

A.1 More ablation experiments analysis of VGR in the main text

In Table 3 (main text) of the main text, the experiment incorporating the complete components of VGR achieved the optimal results. We also list the outcomes when either component is removed—by eliminating grounding cues ("w/o Grounding") or disabling the reasoning process ("w/o Reasoning"). Comparing the results without grounding (using only reasoning data with cropped image feature replay) against the baseline LLaVA-NeXT-7B (first row) demonstrates that even without grounding, the performance exceeds the original baseline, highlighting the rationality and critical role of our reasoning data construction. Additionally, comparing the results without reasoning (retaining grounding with cropped image feature replay) against the LLaVA-NeXT-7B baseline shows that these outcomes also surpass the original baseline, validating the effectiveness and significance of using grounding boxes for image feature replay.

In Table 4a (main text) of the main text, the two rows of experimental results ablate the impact of adding detection loss. The results without detection loss still outperform the LLaVA-NeXT-7B baseline, confirming the validity of the other two components: feature replay and reasoning data. Table 4b (main text) presents ablation results for feature replay; removing feature replay still yields performance superior to the baseline, underscoring the rationale behind the detection loss and reasoning data components. Table 4c (main text) ablates reasoning data, and the results without it still exceed the baseline, demonstrating the effectiveness of detection loss and feature replay.

A.2 More experiments of VGR on different MLLMs

Table 8: **Ablations on different pooling strategies.** Different pooling strides for each type of image are important. We utilize 2×2 for **Base** image feature and **Replayed** image feature, and 4×4 for **Local** images feature. It is noted that above the horizontal line are the results for the original data without VGR-SFT data.

Model	Base	Local	Replayed	Crops	Vtoken	MMStar	ChartQA	DocVQA	TextVQA	InfoQA
LLaVA-Vicuna7B	–	–	–	4	2880	37.6	54.8	77.4	64.9	37.1
LLaVA-Vicuna13B	–	–	–	4	2880	40.4	62.2	77.5	66.9	41.3
LLaVA-Vicuna7B	2×2	4×4	–	4	288	37.5	53.4	52.0	57.0	30.1
LLaVA-Vicuna7B	2×2	4×4	–	20	864	41.3	60.2	71.7	62.7	38.4
LLaVA-Qwen2-7B	2×2	4×4	–	20	864	39.4	49.8	78.2	58.5	39.4
<i>With VGR-SFT data and replay image feature</i>										
LLaVA-Vicuna7B	2×2	4×4	2×2	20	864+100	41.7	67.7	73.7	63.9	39.8
LLaVA-Vicuna7B	2×2	2×2	2×2	20	3024+100	41.7	67.7	73.7	63.9	39.8
LLaVA-Vicuna13B	2×2	2×2	2×2	20	3024+100	44.6	71.7	78.6	64.9	41.8
LLaVA-Qwen2-7B	2×2	2×2	2×2	20	3024+100	46.9	62.7	82.5	61.9	42.5

To illustrate the model performance under different settings, we correct a typo and incorporate additional experimental results in Table 8, including replacing Vicuna-7B with Qwen2-7B on the LLaVA-NeXT 13B and LLaVA-NeXT baselines. We sincerely apologize for the typo in the main text, where the results of the first two rows were incorrectly stated as the raw baseline and VGR setting.

A.3 The Case of VGR

Figure 5 shows a case in VGR-SFT with different formulations, after being processed separately, these three types of data are used to train VGR. The corresponding results are w/o Grounding, VGR-7B and w/o Reasoning in Table 3 (main text).

B Reasoning Data Pipeline

B.1 Details on Data Construction

In this section, we elaborate the details on data curation.

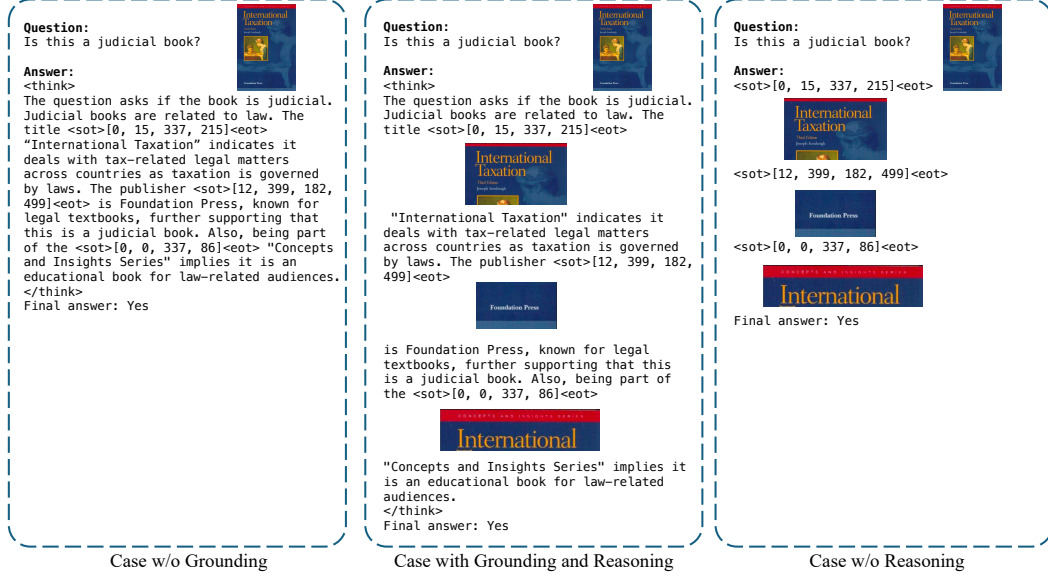


Figure 5: Example of training data in VGR-SFT in different formulations.

Reject Sampling. During the reject sampling, we implement two verification steps with online commercial model, which is Doubao1.5-VL [13] in our implementation. The prompts for remote requests are detailed in Table 9, where two distinct prompts are designed for correctness verification of open-ended problems and grounding area verification, respectively. To process responses from the commercial model, we use a simple parser to convert the output into an integer ranging from 0 to 5. A threshold of 3 is applied to filter out noisy data, ensuring the quality of the dataset.

Data Rewriting. The data rewriting strategy is introduced to address amendable errors. First, during the reject sampling phase, we perform ground-truth-aligned rewriting to reconcile the generated answers with the ground-truth annotations in our training data. To avoid an absurd change in "Final Answer", we use the commercial model again to fix the reasoning chain. Second, prior to training the reasoning model, we introduce a comprehensive rewriting for all reasoning data, ensuring all data matches the same format, mitigate confusion in the reasoning chains, reduce information leak before the replay and avoid failed answer extraction. The prompts are shown in Table 9

Annotator Training. We train our annotator with two type of data, cold-start data that includes visual reasoning generated by previous steps, and Open-R1 data [8] from pure-text reasoning chain distilled from Deepseek-R1 [12]. We use the learning rate of $1e - 5$, batch size of 128 and trained the model for 6000 steps. During annotator training, we use different prompts for these two types of data, which reduce confusion for the model, the prompt is shown in Table 9. As shown in the next paragraph, we use the combination of these two prompts to generate more diversified answers.

Annotation Generation. We employ distinct prompts to guide our cold-start model and annotation model in generating training data, as outlined in Table 10. For the cold-start model, we provide a highly detailed prompt with an illustrative example to ensure data quality and format consistency. For the annotator, we use a hybrid prompt that integrates visual grounding and Open-R1 prompts, enabling the generation of complex multi-step reasoning akin to DeepSeek-R1’s behavior.

B.2 Visualization of Data and VGR

We show examples from our data pipeline in Figure 6. As shown in the figure, our data curation pipeline is able to improve the data quality step-by-step, the annotator improves the reasoning depth and annotation efficiency, while the training data from the rewriting is more concise and easy to learn.

In Figure 7, we visualize the responses of VGR on the MMStar and ChartQA benchmarks. VGR automatically and accurately locates target regions in the responses, generates correct reasoning based on the content within these regions, and ultimately provides accurate answers.

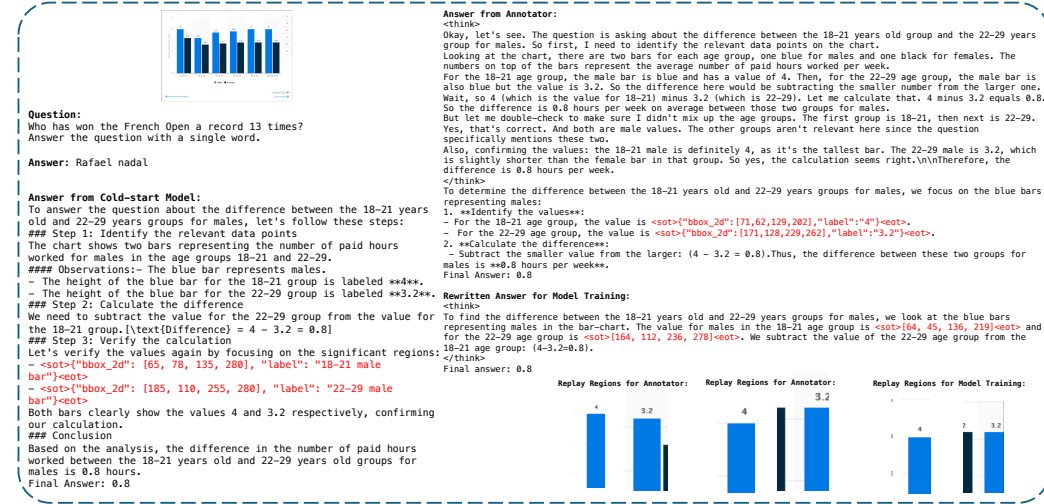


Figure 6: Example of data from original data, cold-start model, annotator and training set.

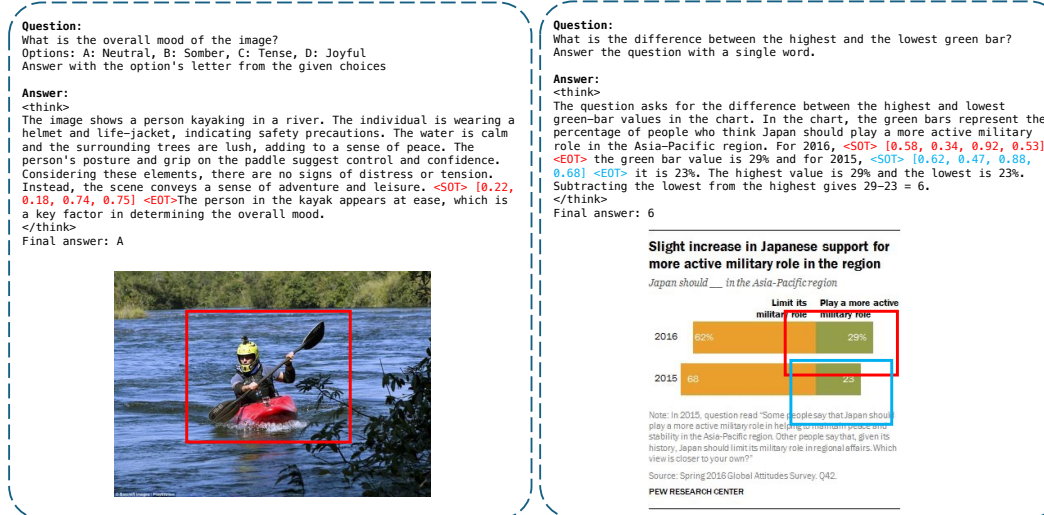


Figure 7: Example of VGR response in MMStar and ChartQA benchmarks.

Table 9: Prompt for **VGR** reject sampling and data rewriting.

Stage	Prompt
<i>Reject Sampling</i>	
Correctness Verification	You are an annotator, your goal is to check if the reasoning process is aligned with multimodal question and answer. You will be given the question, ground truth, the reasoning chain and the original answer. Output an integer from 0 to 5: output 5 if the reasoning chain is aligned with the ground truth (even if the answer has some mistakes), output 0 if the reasoning chain is not aligned with the ground truth. Question: {question} Ground truth: {gt} Reasoning chain: {answer} Original answer: {final_answer}
Visual Grounding Verification	You are an annotator, your goal is to check if the short content description of the bounding box is aligned with the image. I will send you two images: one is the original image and the other is the bounding box area cropped from the original image. Output a integer from 0 to 5, 0 means the content is not aligned with the content, 5 means well aligned. Check if the content from the second image is "{content}".
<i>Data Rewriting</i>	
Ground-Truth Rewriting	You are an annotator, your goal is to check if the reasoning process is aligned with multimodal question and answer, and rewrite the reasoning chain and the answer to match the ground truth. You can add more details to the answer, but all information introduced by the ground truth should be covered. You will be given the question, ground truth, and the original answer with the reasoning for reference. Output the answer with the reasoning process: think first, then answer the problem. The final answer that matches the ground truth should be written after "Final answer:". Question:{question} Ground truth: {gt} Answer with Reasoning: {answer}
Reasoning Chain Rewriting	You are an annotator. Your goal is to check whether the reasoning process aligns with the multimodal question and answer, and rewrite the reasoning chain and the answer to match the ground truth. You can add more details to the answer, but it must cover all the information provided by the ground truth. You need to remove any redundant, confusing, or incorrect information from the original answer. The rewritten answer should be logical and concise. The answer should follow a strict format: all the thinking parts should be enclosed within <think></think> tags, and then state the ground truth starting with "Final answer:". All location information should be enclosed within <sot><eot> tags; the content of <sot><eot> includes "bbox_2d" and "label", which are simply copied from the original answer and should NOT be changed. You need to reference the area before mentioning any information in the area, and each location should be mentioned only once (i.e., duplicate <sot><eot> tags with the same information should be removed). You will be provided with the question, the ground truth, and the original answer with its reasoning for reference. Output the answer along with the reasoning process, make the answer fluent, and do not use the ground truth in the reasoning process. You must reference at least one location with <sot>...<eot>, the content of <sot><eot> is copied exactly from the original answer. Think through the problem and the referenced area, and then write the final answer that matches the ground truth after "Final answer:". Only return a single-line final answer, which should strictly conform to the ground truth. Question: {question} Ground truth: {gt} Answer with Reasoning: {answer}

Table 10: Prompt for **VGR** model training and data construction.

Stage	Prompt
Annotator Training	
Cold-start Data	Think step by step and answer the following question, you need to reference the key area with <code><sot>{"bbox_2d":[x1,y1,x2,y2],"label":"..."}</code><code><eot></code> bounding box format and give the final answer with "Final answer:". The size of the image is {image.width} x {image.height}. {original_question}
Open-R1 Data	{original_question} Give step by step reasoning before you answer. This requires engaging in a comprehensive cycle of analysis, summarizing, exploration, reassessment, reflection, backtracing, and iteration to develop well-considered thinking process. You need to use <code><think></code> <code></think></code> to wrap your reasoning process and answer the final answer enclosed in LaTeX's <code>\boxed</code> tag.
Data Annotation	
Cold-start Model	You must locate and focus on the major objects that significantly contribute to solving the question. Prioritize the output of bounding boxes for larger and more significant areas, minimizing the inclusion of smaller, less relevant regions. Output the bounding box coordinates of these key objects in JSON format. As you reason step by step, ensure each step includes detailed considerations such as analyzing the question, summarizing relevant findings, brainstorming new ideas, verifying the accuracy of the current steps, refining any errors, and revisiting previous steps. During this process, emphasize larger and more important areas using the bounding box format <code><sot>{"bbox_2d":[x1,y1,x2,y2],"label":"..."}</code><code><eot></code> to reference visual details and information. Reference the area before mentioning its content. Finally, answer the question with "Final Answer: xxx". For example: To answer the question [state the question here], first, we need to identify [describe what needs to be identified], let me focus on this significant region <code><sot>{"bbox_2d":[x1,y1,x2,y2],"label":"..."}</code><code><eot></code>. You need to replace x1, y1 with the actual pixel coordinates. In this region, I observe [describe what you see in the region, such as the letter xxx]. This observation indicates [explain the significance of what you saw]. Based on this analysis, we can conclude that [continue with the reasoning process]. Therefore, the answer is [state the answer]. Final Answer: [answers] Now, I will provide you with a Question. Please output the answer with the bounding box incorporated into the reasoning as described above, focusing on larger and key areas, and minimizing small or irrelevant boxes.' {original_question}
Annotation Model	Think step by step and answer the following question, you need to reference the key area with <code><sot>{"bbox_2d":[x1,y1,x2,y2],"label":"..."}</code><code><eot></code> bounding-box format and give the final answer with 'Final answer:'. The size of the image is {image.width} x {image.height}. {original_question} Give step by step reasoning before you answer. This requires engaging in a comprehensive cycle of analysis, summarizing, exploration, reassessment, reflection, backtracing, and iteration to develop a well-considered thinking process. You need to use <code><think></code> and <code></think></code> to wrap your reasoning process start and end. During reasoning, reference the key area with <code>["bbox_2d":[x1,y1,x2,y2],"label":"..."]</code> only at the thinking process. Do not include box information in the final answer. Ensure the final answer appears only once and contains only the solution or conclusion.
Reasoning Model Training	
VGR-SFT Data	Think step by step and answer the following question, you need to reference the key area with <code>"<sot>[x1,x2,y1,y2]<eot>"</code> bounding-box format and give the final answer with "Final answer:". {original_question}